

A fixed-parameter tractable algorithm for matrix domination

Mark Weston

Department of Computer Science, University of Victoria, P.O. Box 3055, Victoria, BC, Canada

Received 5 December 2001

Communicated by F. Dehne

Abstract

MATRIX DOMINATION is the NP-complete problem of determining whether a given $\{0,1\}$ matrix contains a set of k non-zero entries that are in the same row or same column as all other non-zero entries. Using a kernelization and search tree approach, we show the problem to be fixed-parameter tractable with running time $\mathcal{O}(n^3 + 1.959^k k^{5/2})$.

© 2004 Elsevier B.V. All rights reserved.

Keywords: Computational complexity; Parameterized complexity; Graph algorithms; Analysis of algorithms

1. Introduction

MATRIX DOMINATION is the problem of whether or not a given $\{0,1\}$ matrix contains a set $D = \{d_1, d_2, \dots, d_k\}$ of k 1's such that for each other 1 entry, some entry of D is in the same row or the same column. A *matrix dominating set* is a set of k entries with this property. We say that a particular non-zero entry (i, j) of M is dominated *row-wise* (*column-wise*) if there are no entries in the matrix dominating set in the same column (row) as (i, j) ; i.e., there must be a non-zero entry in the dominating set in the same row (column) as (i, j) .

In our graph theoretical analysis, we deal exclusively with undirected graphs not containing multiple edges. In a graph $G = (V, E)$, two vertices that are connected by an edge are *adjacent*, and a vertex *dominates* its adjacent edges and vertices as well as itself.

If an edge is connected to a vertex, that edge is *incident* to that vertex. Two edges that have a vertex in common are called *adjacent*. A set of edges in a graph is called a *matching* if no two of its elements are adjacent. A *complete* graph (or subgraph) is a graph (or subgraph) in which all vertices are pairwise adjacent. We use the concept of a *bipartite* graph; here, the vertex set can be partitioned into two sets V_1 and V_2 such that every edge (u, v) is between some u in V_1 and v in V_2 , i.e., every edge goes between the two sets.

Yannakakis and Gavril prove MATRIX DOMINATION is NP-complete in [10, p. 365], and describe applications in the context of edge dominating sets. They propose the transformation of the problem to a bipartite graph that we use in Section 3. Several other applications are provided in [10,6]: the problem is related to edge dominating sets which are relevant to telephone switching applications. The problem is found in computer chip fabrication and reconfigurable programmable logic arrays, and the spare allocation problem described in [6] is essentially the same problem.

E-mail address: mweston@uvic.ca (M. Weston).

Section 2 further introduces the problem and provides some material on fixed-parameter tractability. In Section 3, we present a kernelization, or reduction of the problem to a size bounded by k , and the algorithm to determine whether or not the problem instance has a solution. In Section 4 we outline a complexity analysis of the given algorithm.

2. Preliminaries

The problem of MATRIX DOMINATION is defined as follows:

Input: An $n \times m$ matrix M with entries from $\{0, 1\}$, and a positive integer k .

Parameter: k .

Question: Is there a set of k or fewer non-zero entries in M that dominate all others? Formally, does there exist a subset D of $\{1, 2, \dots, n\} \times \{1, 2, \dots, m\}$ with $|D|$ no greater than k such that M_{ij} is non-zero for all (i, j) in D and such that for any non-zero entry (i, j) in M , there is an element (i', j') in D so that either $i = i'$ or $j = j'$?

For example, in Fig. 1, the two circled non-zero items dominate all other non-zero items (as marked by the shading).

Note that M does not have to be a square matrix. The problem is NP-complete [7] by a transformation from MINIMUM MAXIMAL MATCHING in [10].

In this paper we explore the complexity of the problem with respect to the recent research in fixed parameter tractability (see Downey and Fellows [2]). Our proof is simpler than previous (unpublished) results [5]. A parameterized problem with problem size n and parameter k is defined to be *fixed-parameter tractable* (FPT) if it can be solved in time $f(k)n^c$ or $f(k) + n^c$, where c is a constant independent of both

0	1	0	0	1	0
1	1	1	0	1	0
0	1	0	0	0	0
0	1	0	1	1	1

Fig. 1. Example of MATRIX DOMINATION.

n and k and $f()$ is an arbitrary function. Thus the classical exponential complexity explosion which we see for NP-complete problems can often be confined to the a parameter k in the function $f()$. Typically, FPT problems result in small and manageable functions for $f()$, often in the range 2^k , and c is typically small as well.

A good example is the NP-complete problem VERTEX COVER, which is the problem of whether a graph G contains k vertices such that every edge contains at least one of the k vertices as one of its endpoints. It yields a simple FPT algorithm of running time $\mathcal{O}(2^k n)$ (by a proof in [3, Theorem 2.1]; see [4] for more on VERTEX COVER and FPT). Thus VERTEX COVER is, even by this early FPT algorithm, well-solved for reasonably small k , which yields a reasonable running time. At the time of writing the fastest result is an algorithm running in time $\mathcal{O}(kn + 1.285^k)$ [1].

This example and many others suggest that reasonable parameterized functions are obtainable for many natural problems; MATRIX DOMINATION is one of a small family of problems with parameterized running times with exponent base less than 2.

3. Kernelization and algorithm

We now introduce the method of creating a *kernel*, by reducing the problem instance I to an equivalent instance I' , where the size of I' is bounded by a function of the parameter k . This is a polynomial time parametric transformation of MATRIX DOMINATION to itself. Recall that the initial matrix M is bounded in size by n with k as the parameter.

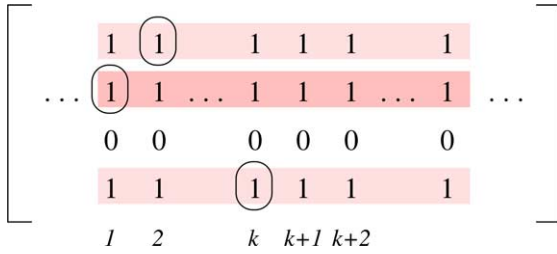
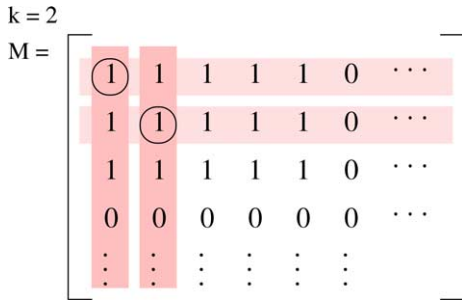
The algorithm on the input matrix M proceeds as follows.

Step 1: For all rows r and columns c in M , if r or c is “empty” (i.e., contains only zero entries) it can be deleted from M to produce a matrix M' .

This is obvious as zero entries neither have to be dominated nor can be part of the dominating set.

Step 2.0: For all columns c (or similarly rows r), if there are at least $k + 2$ identical copies of c the columns can be reduced to $k + 1$ identical copies of c .

This is based on the following observation: if M' can be dominated by a set of 1's of size at most k then at most k copies of c can contain an entry from the dominating set. The remaining copies must

Fig. 2. A matrix with at least $k + 2$ identical copies of a column.Fig. 3. M has at least k copies of a column with at least k non-zero entries.

be dominated by non-zero entries in rows (whether the dominating entries are part of the $k + 1$ copies or not is unknown). Thus only $k + 1$ copies of c are required to constrain the solution to covering the non-zero entries in c row-wise. This is illustrated in Fig. 2, where the shading indicates that the non-zero entries in the copies of c must be covered row-wise.

Step 2.1: If there are greater than k copies of a column (or row) with greater than k non-zero entries, we can immediately answer “no” for the given problem instance (we say the instance is a no-instance).

This is because a column with greater than k non-zero entries must have a non-zero entry in it to dominate the extra non-zero entries, and thus if there are greater than k columns like this, then we must have greater than k non-zero entries to dominate them, so the input is a no-instance. Fig. 3 illustrates this.

At this point the size of the matrix is now bounded by a function of k . To see this, consider that if the matrix has a solution the rows and columns can be rearranged so that the at most k dominating non-zero entries are in the top left corner of the matrix (see Fig. 4). Then the bottom right corner of the matrix must be zero (as otherwise the k entries would not

dominate them). Thus after the reduction there can be at most $2^k - 1$ unique rows (as all-zero rows have been removed) (similarly for columns), each of which can be repeated at most $k + 1$ times, giving an upper bound on the largest dimension of the matrix of $(2^k - 1) \times (k + 1) \in \mathcal{O}(2^k k)$. M' is our problem kernel.

Once we have a kernelization of the problem, we use an algorithm for the following problem of CONSTRAINT BIPARTITE VERTEX COVER:

Input: An undirected bipartite graph $G = (V_1, V_2, E)$ and two integers k_1 and k_2 .

Question: Do there exist two subsets of vertices $C_1 \subseteq V_1$ and $C_2 \subseteq V_2$ with $|C_1| \leq k_1$ and $|C_2| \leq k_2$ such that each edge in E has at least one of its endpoints in $C_1 \cup C_2$?

To use an algorithm for this as a black box, we need to express the problem kernel M' as a graph appropriately.

Step 3: Suppose by the previous reductions the reduced matrix M' has m columns and n rows with all rows and columns non-empty.

We model the matrix M' as a bipartite graph as follows: let M' be the incidence matrix of a bipartite graph $G = (R, C, E)$; we create two disjoint node sets R and C where $|R| = n$ and $|C| = m$. For every row i in M' , create a node r_i in R , and for every column j in M' create a node c_j in C , and for every non-zero entry M'_{ij} in M' create an edge (r_i, c_j) . The graph is bipartite as all edges are between nodes in R and nodes in C . Fig. 5 provides an example.

At this point we need to find a set of edges of size at most k whose endpoints form a vertex cover of the graph. To do this we find all vertex covers and then test to see if there is a set of k edges that connects them.

Step 4: Using a search tree, we find all vertex covers of size at most $2k$ for the input. This step is explored in detail in [6], and for brevity's sake we omit the details. Intuitively we can expect the search tree to be of size $2^{2k} = 4^k$ but using kernelization techniques based on vertex degree, the size is reduced to $1.399^{2k} = 1.959^k$. The algorithm presented in [6] uses a search tree approach after restricting the input by considering vertex degrees of the nodes. Briefly, a node v with $\text{degree}(v)$ at least k must be included in the vertex cover because if not, all of the more than k nodes adjacent to v must be included, which is not possible.

$\leq k$ dominating entries

 $\leq 2^k - 1$ different lines, each
repeated $\leq (k + 1)$ times

1	...	1	1	0	0	1
1	...	1	1	$\vdots$	$\vdots$	$\vdots$
		$\vdots$		0	1	...
1	...	1	1	1	0	0
0	...	0	1	all zero		
0	...	1	0			
		$\vdots$				
1	...	1	0			

Fig. 4. Necessary configuration for a yes-instance of the problem, and its size.

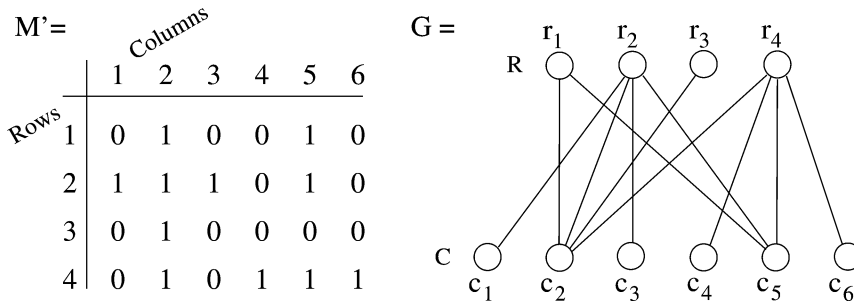


Fig. 5. Matrix to bipartite graph conversion.

The problem can thus be reduced further as then each node is limited to degree k , and thus the graph considered in further stages in [6] has at most $2k^2$ edges and at most $2k^2 + 2k$ vertices. This step then generates all vertex covers of the bipartite graph with $2k$ vertices. It is important to note that the algorithm in [6] generates all possible solutions conforming to the required parameter, as we now restrict this algorithm so that any vertex cover that uses at least k edges from either edge set R or C is eliminated. This can be done in one pass, adding only a linear term to the time complexity, which is dominated by the other terms in the complexity analysis.

Step 5.0: We now have a set $S = \{G' : G' \text{ is a vertex cover with at most } k \text{ vertices in each of the two edge sets } R, C\}$. The graphs in S have at most $2k$ vertices and at most k^2 edges. The problem is now one instance of EDGE COVER restricted to bipartite graphs:

Input: A bipartite graph $G = (V_1, V_2, E)$, and an integer k where $|V_1| \leq k$ and $|V_2| \leq k$.

Parameter: k .

Question: Is there a set of edges $E' \subseteq E$ with $|E'| \leq k$ such that for all v in $(V_1 \cup V_2)$ there is an edge $e \in E'$ such that v is incident to e ? Informally, can we find up to k edges in G that connect to all of the vertices in G ?

See Fig. 6 for an example of the problem at this stage. The graph contains two edges that connect the nodes in the vertex cover.

Since the graph G has $|V_1| \leq k$ and $|V_2| \leq k$ the instance can be solved naively in at most k^k time by trying all possible combinations of edges. However, polynomial-time algorithms exist for the general problem EDGE COVER (see [9]). The algorithm that follows works in time $k^{5/2}$ on the bipartite graphs as we see in the complexity analysis in Section 4.

Step 5.1: Without loss of generality, assume $|V_1|$ is at most $|V_2|$ and both are at most k . Let $|V_1| = x$. In polynomial time, determine if a matching exists between the x vertices of V_1 and any x vertices of V_2 . To find the maximum bipartite matching in a

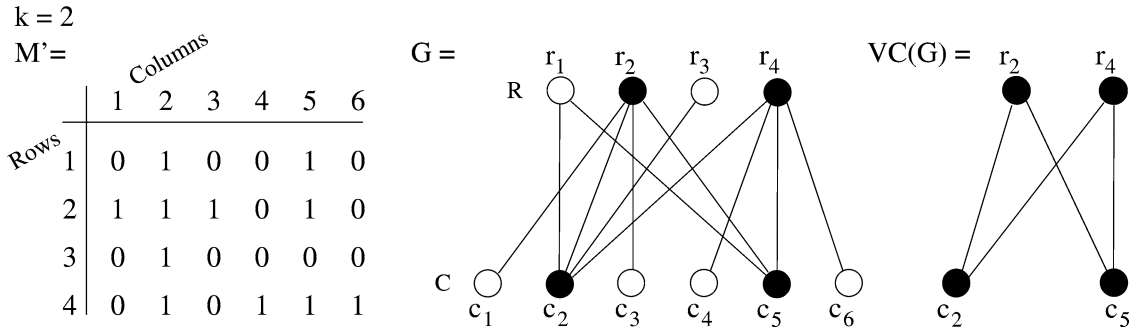


Fig. 6. Example of VERTEX COVER with yes-instance.

graph, several fast algorithms exist which run in small polynomial exponents of the size of the vertex set; we use the $n^{5/2}$ algorithm of Hopcroft and Karp [8]. This matching uses x edges of G , with x at most k . At this stage, if no matching exists, the problem is immediately a no-instance and we can answer no.

Step 5.2: Mark the x nodes used out of V_2 used in this matching. There are now at most $k - x$ nodes unmarked in V_2 . If the graph is a yes-instance, there must be edges (one per node) that connect these remaining $k - x$ nodes to nodes in V_1 (which have all been connected already to V_2 , so it does not matter which node the unmarked nodes in V_2 connect to).

Step 5.3: After considering the remaining vertices, if there are no degree-zero vertices the problem is a yes-instance, otherwise it is a no-instance.

At this point, if the problem is a yes-instance, we have found a maximum matching between the x nodes of V_1 and any x nodes of V_2 , using x edges. Then we have used at most $k - x$ edges to connect the remaining nodes in V_2 to nodes in V_1 . Thus we have used at most $k - x + x = k$ edges to connect every node in V_1 to a node in V_2 , and vice versa.

If the answer to the kernelized problem is no, the problem instance M is a no-instance, and if the answer is yes, M is a yes-instance.

4. Analysis

It is clear from the construction that a solution to the graph problem of determining a vertex cover of size at most $2k$ is a solution to the original instance of MATRIX DOMINATION. We show that the solution to the matrix problem is a solution to the graph problem.

A solution S of an instance of MATRIX DOMINATION M consists of a set, of size at most k , of non-zero entries of M such that for any non-zero entry x in M that is not in S , there is a non-zero entry in S in either the same row as x or same column as x . The algorithm above transforms S into a set of (row, column) pairs Q in the bipartite graph G formed by the construction; these pairs must all be edges as each (row, column) pair with $row \in R$ and $column \in C$ denotes the location of a non-zero entry in M .

Now consider the non-zero entries in M that are not in S , call them X . An entry (i, j) in X is transformed into an edge (i, j) in the bipartite graph G . As S is a solution set, every non-zero entry (i, j) in X has either $i = r$ or $j = c$ for some (r, c) in S . As (i, j) is transformed into an edge in G , (i, j) must share one endpoint with $(r, c) \in Q$. Thus every non-zero entry is mapped into an edge which shares an endpoint with an edge in Q . Thus the endpoints of the edges Q form a vertex cover of G as every other edge shares an endpoint with some edge in Q . As G is bipartite the vertex cover consists of at most k vertices from R and k from C , and the k edges in Q make up a solution to the graph problem.

4.1. Complexity

Without loss of generality we assume M is $n \times m$ with n at least m . In Section 3 we first remove empty rows and columns, which is trivially done in time $\mathcal{O}(n^2)$. Removing $k + 1$ duplicates of columns takes time $\mathcal{O}(n^3)$. Converting the matrix M' into a bipartite graph, takes time $n + n^2 \in \mathcal{O}(n^2)$ as every row and column and then every entry in the matrix must be examined. Using induction the size of the search tree

used for determining the vertex cover of size $2k$ can be shown to be bounded by $1.399^{2k} = 1.959^k$; for more detail see [6].

For each vertex cover produced by Step 4, Step 5 takes the following steps: Step 5.1 (determining if a matching exists) takes time bounded by $\mathcal{O}(x^{5/2})$ for determining the maximum matching, where x is the number of vertices in the bipartite graph. Steps 5.2 and 5.3 take time linear in $k - x$. The value of x is at most $k - 1$ as one of the vertex sets may have only one vertex. Thus Step 5 takes time bounded by $\mathcal{O}(k^{5/2})$.

Thus we have an upper bound of

$$\mathcal{O}(n^2 + n^3 + n^2 + kn + 1.959^k k^{5/2}).$$

As k is at most n this is $\mathcal{O}(n^3 + 1.959^k k^{5/2})$.

Acknowledgements

We gratefully acknowledge the assistance of Mike Fellows, who originally suggested the problem and provided valuable discussions. Wendy Myrvold and Ulrike Stege also provided valuable assistance and proofreading. An anonymous referee provided comments which greatly aided the presentation.

References

- [1] J. Chen, I.A. Kanj, W. Jia, Vertex cover: further observations and further improvements, *J. Algorithms* 41 (2) (2001) 280–301.
- [2] R.G. Downey, M.R. Fellows, *Parameterized Complexity*, Springer, Berlin, 1999.
- [3] R.G. Downey, M.R. Fellows, Parameterized computational feasibility, in: P. Clote, J. Remmel (Eds.), *Feasible Mathematics II*, Birkhäuser, Boston, MA, 1995, pp. 219–244.
- [4] R.G. Downey, M.R. Fellows, U. Stege, A framework for systematically confronting computational intractability, in: *Contemporary Trends in Discrete Mathematics* (Stirín Castle, 1997), in: DIMACS Ser. Discrete Math. Theoret. Comput. Sci., vol. 49, Amer. Math. Soc., Providence, RI, 1999, pp. 49–99.
- [5] M.R. Fellows, Personal communication, 1999.
- [6] H. Fernau, R. Niedermeier, An efficient exact algorithm for constraint bipartite vertex cover, in: *Mathematical Foundations of Computer Science 1999* (Szklarska Poreba), in: *Lecture Notes in Comput. Sci.*, vol. 1672, Springer, Berlin, 1999, pp. 387–397.
- [7] M.R. Garey, D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-completeness*, Freeman, New York, 1979.
- [8] J. Hopcroft, R.M. Karp, An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs, *SIAM J. Comput.* 2 (4) (1973) 225–231.
- [9] E.L. Lawler, *Combinatorial Optimization: Networks and Matroids*, Holt, Rinehart and Winston, 1976.
- [10] M. Yannakakis, F. Gavril, Edge dominating sets in graphs, *SIAM J. Appl. Math.* 38 (3) (1980) 364–372.